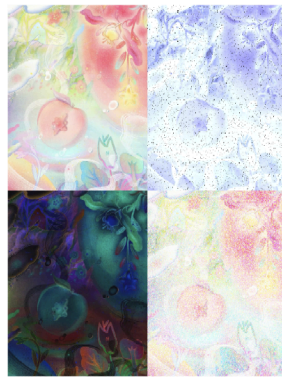


Kimberly By Goytia  
Computational Linear Algebra  
Becca Thomases and Elaine Gorom  
March 28, 2025

# Midterm Write-up

## Warhol Images:



For my first Warhol image, I wanted to try out adding texture or noise and color filtering. In the top right image, I created noise that I then overlaid and subtracted from the original picture. This caused it to have negative space or black dots. I then color-filtered it so that it could have a pale blue glow.

The bottom left corner was playing with the level of color inversion. In the lab, we used 255 for the inversion depth amount, but I decided to change it to 230 and then 270. I found that 230 was too dark and did not allow me to see the different colors highlighted by the 255 inversion. From this point, I went above 255 to see if it would brighten up the image, and it did.

The final image in this initial Warhol image also exclusively includes noise. Unlike the first noise-textured image, I did not change the color filtering or create negative space with the dots. Instead, I left the coloring alone and overlaid the noise. I also tried to see what different noise levels looked like, with 90 being barely grainy and 300 (the quantity I ultimately chose) creating a snowy effect.



In the second Warhol image, I decided to experiment with inversion, grayscale, color filtering, and exposure. The top right picture was first inverted with a depth of 280. After the inversion, I then color-filtered it blue, which surprisingly turned it into a beautiful grayscale image with pink peaking through, making it look almost pearlescent. The bottom left image was an attempt at trying to color filter the entire image to purple, but it turned out to be more on the pink side of the purple scale. Finally, the bottom right image experienced increased contrast for the pink/red shades in the image and increased white exposure.



My final Warhol image continued with color filtering, but I also started to implement flipping, rotating, and decreasing brightness. For the top right image, I used the `fliplr()` function to flip the image from left to right so that it could mirror the original picture. I also filtered it with our lab's sepia filter to add an interesting vintage look. The bottom left was flipped with the regular `flip()` function that flips an image from top to bottom. This way, I would be able to create another reflection effect. I also changed the brightness of the image so it would be darker. Lastly, for the bottom right image, I once again changed the orientation of the image, but I used `rotate` this time, where I rotated the image 180 degrees. I then color-filtered it so that it turned everything that was not dark pink into cyan and highlighted the dark pink.

## Code questions:

- **Q1: Explain the purpose of uint8 and double commands in the code above.**
  - The double command converts the pixel color values in the processed image from uint8 to double. Then, unit8 converts the image back to the original pixel format.



- **Q2: What does the transformation above do?**
  - The transformation that filters the image using the FilterMatrix =  $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$  increases the red color in the picture to saturate it in a bold red.



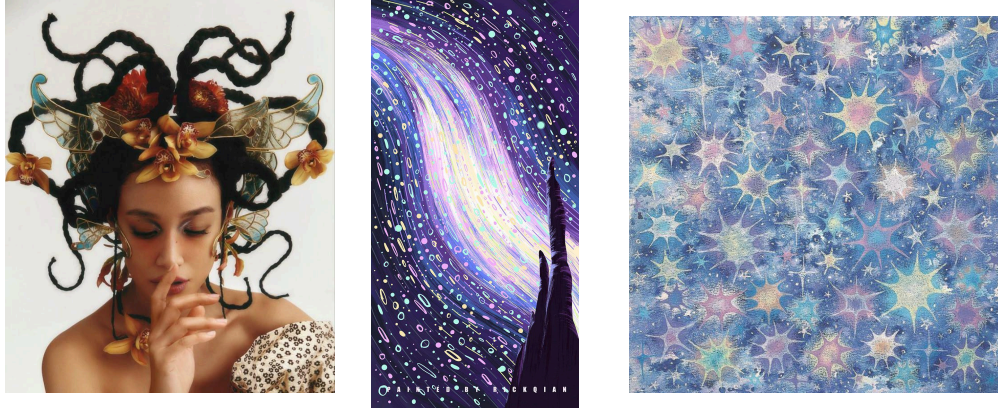
- **Q3: What does the transformation above do?**
  - The transformation that filters the image using the ModifiedMatrix =  $\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$  changes the red hues in the image to blue hues.
- **Q4: What do each of these commands do?**
  - ImJPG(100:m-100, 100:n-70) - Crops the image, but for some reason, it also comes out in gray scale.
  - flip(ImJPG) - Flips the image so what was on the bottom is now on top.
  - rot90(ImJPG) - Rotates the image 90 degrees so the apple that was in the bottom left corner vertically is now on the bottom right corner horizontally.
  - 255-ImJPG - Shows the color inversion of the image.
  - ImJPG -50 - Mutes or darkens the color of the image.
  - uint8(1.25\*ImJPG) - Changes the brightness/exposure of the image.



- **Q5: Explain what the initial transformation does to the image. What does the inverse transformation do with the colors?**
  - Information is actually not lost at least from what I am seeing. The image went from being dull with the matrix A transformation to having its normal coloring again after the inverse transformation.

## Free Section:

For the free section, I decided to experiment with edge detection, color segmentation, and image overlay. In the matlab file I have detailed the resources that I used to help with piecing together the final two images that encapsulate all three ideas together.

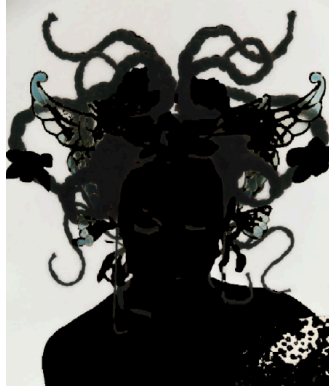


First, I began by reading in the three above images. The foreground will be the person and two background images which I will use in different overlay scenarios.

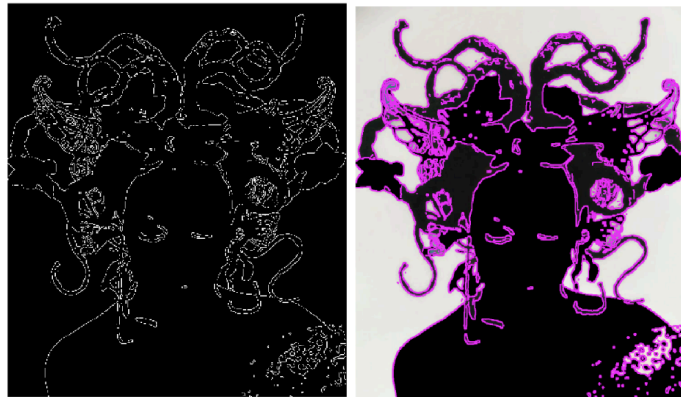


Once I had my images, I then used color segmentation on the person's picture. To do this, I made 3 cluster color groups and used the built-in MATLAB function: `imsegkmeans()`. The function `imsegkmeans()` in matlab segments an image into k color regions by using k-means clustering. It takes an RGB image and returns an image with the color pixels categorized into distinct clusters, as seen above. It can also be improved by using Gabor filters, which is shown in the MATLAB documentation, but I consider that an extension to the project if I continue this work in a future project.

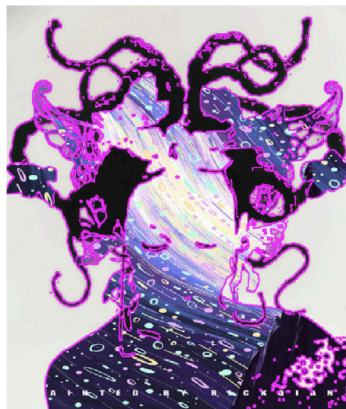




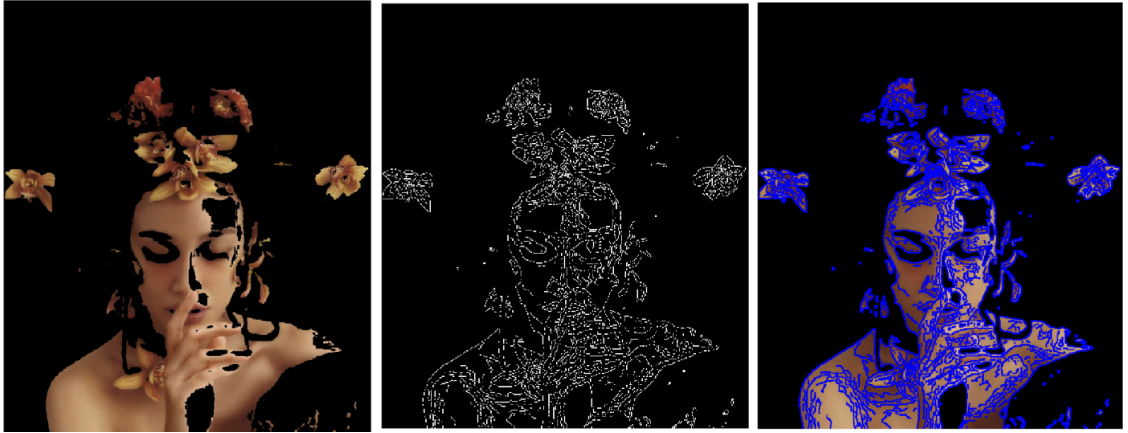
Then, I turned the image into a mask where any segment that was not assigned yellow would be removed from the image. This would allow me to place the background image inside of the person's outline.



After I had the mask of the person, I passed it through edge detection to highlight the person's outline. For edge detection, I used the function `edge(image, 'canny')`, which performs Canny edge detection. With the edge detection, I also used the built-in function `imdilate()`, which helps make the edges thicker which increases their visibility for when I went in to change the color.



Finally, I overlaid the masked edge picture with the first space background image. To do this, a for loop was used to replace the black segments of the masked picture with the corresponding channels in the background image.



I later made a second image that follows the same methods of color segmentation, except this time I kept the section that was labeled as 2 or cyan. Everything that was not cyan was removed from the image. I then edge-detected the remaining masked image and changed the edge color to blue.



Once I had the edge-detected image, I passed it through the same for loop that replaces the black pixels with the second space background image. This created my final overlaid edge image.